Day 174: Reproducibility: Week 25's Build

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 25, Day 7 | Complete study guide

Watch: Reproducible machine learning and science with python (Prabhant Singh, PyData) | Using Reproducible Experiments To Create Better Machine Learning Models (PyData Global 2021)

Script: `day174_reproducible_study.py` · every number in this guide is copied by program from `day174_run_output.txt`, the real output of that script.

One tuning study built from all six of this week's parts: Day 170's finder sets the learning-rate range, Day 168's random trials and Day 169's Bayesian optimiser choose configurations, Day 172's folds and Day 171's early stopping score them, and Day 173's tracker logs every step. It was then run again and again, each time in a new Python process. The same command reproduced 21 of 21 steps to the bit, and so did a study that crashed and resumed from its log, after a first draft of the resume failed this check. Another seed chose different settings and a test loss of 0.3364 instead of 0.3331; folds with no seed made the same command give two different studies; one BLAS thread instead of four changed the last digits of every score but none of the choices.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

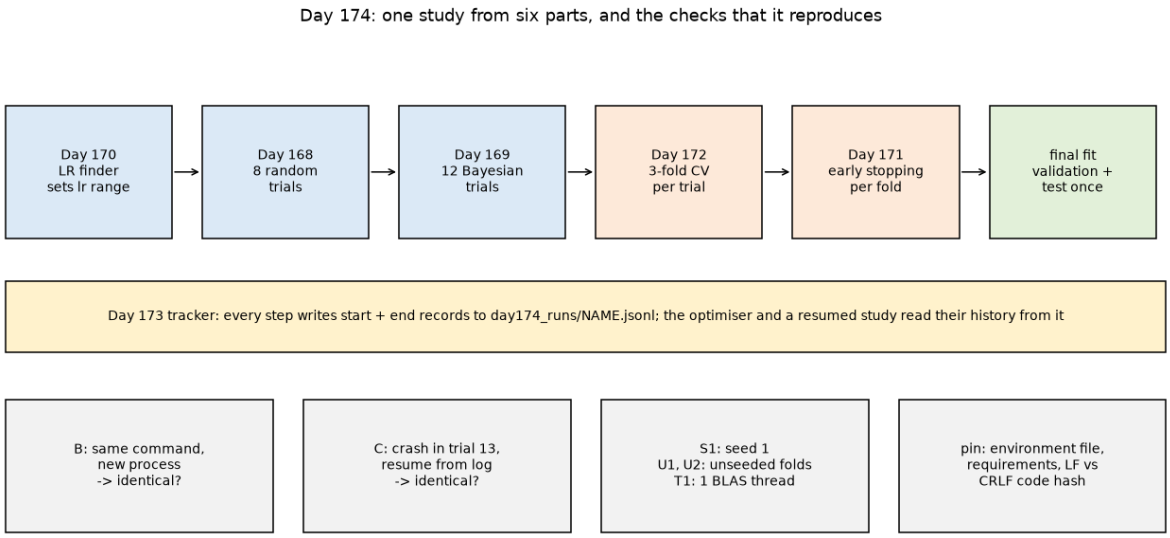
A result is **reproducible** when the same code, run on the same data in the same environment, gives the same numbers. **Bit-identical** is the strict form: every digit the same. It needs every source of randomness seeded, nothing that depends on timing or on the order a computer happens to add numbers in, and the environment (Python, library versions, thread counts) held fixed. A weaker form, **statistically reproducible**, only asks that a rerun lands within the noise; that needs the noise measured.

Pinning the environment means recording exact versions (a requirements file) and the settings that change results, and checking them before trusting a rerun.

REAL-WORLD SCENARIO

A bakery chain's recipe. The same recipe in a different shop should give the same loaf; when it does not, the cause is an unrecorded ingredient: a different flour, a hotter oven. Reproducibility is writing down every ingredient, then baking the loaf twice to prove nothing was missed.

SYSTEM DIAGRAM



The study's pipeline (top), the tracker every step writes to (middle), and the reproducibility checks (bottom).

Data: Week 24's split of UCI Adult. The study tunes on 3,000 training rows with 3 folds, using a separate 6,000-row stop set for early stopping. The chosen settings are retrained on all 3,000 and scored once on the 6,032 validation rows and once on the official test file (15,060 rows), which no part of Week 25 had touched and which chooses nothing. Each study runs in its own child process, started with BLAS threads fixed at 4 and PYTHONHASHSEED at 0.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. The study: six parts, one tracked log

DEFINITION — start here, no prior knowledge assumed

The finder runs 300 steps of rising learning rate on a fresh network and suggests one tenth of the rate with the lowest smoothed loss. The learning rate is then searched on a log scale from 300 times below that suggestion to 3 times above it; patience from 1 to 20; batch size from 32 to 256.

Eight trials are random points (Day 168), twelve are proposed by expected improvement (Day 169) from the trials before them, read back from the log. Every trial's score is its mean held-out log loss over 3 folds (Day 172), each fold trained with early stopping (Day 171). The finder, every trial and the final fit are tracked runs (Day 173).

REAL-WORLD SCENARIO

A production line where every station stamps the part it handled: the finished product carries its whole history.

IMPLEMENTATION

```
def study(name, seed, crash_at=None, unseeded=False, below=LR_BELOW):
    os.makedirs(RUNS, exist_ok=True)
    T = d173.Tracker(os.path.join(RUNS, name + ".jsonl"))
    D = load_data()
    data = d173.fingerprint(*[D[k] for k in sorted(D)])
    code = d173.file_hash(__file__)
    fp = dict(step="lr_finder", steps=FIND_STEPS)
    found = d173.query(T.runs(), where=lambda r: r["params"] == fp)
    if not found:
        with T.run(fp, seed, data, code) as r:
            lrs, raw, sm = d170.lr_find(d170.mlp_stepper(D["Xc"], D["yc"], d126.Adam, seed),
                d170.lr_range(d170.LO, d170.HI, FIND_STEPS))
            r.summary.update(d170.suggest(lrs, sm), lrs=lrs.tolist(), sm=sm.tolist())
        found = d173.query(T.runs(), where=lambda r: r["params"] == fp)
    space = space_from(found[0]["summary"]["min_over_10"], below)
    folds = d172.shuffled_folds(CV_ROWS, K, None if unseeded else seed)
    U0 = d168.random_unit(N_RANDOM, len(space), np.random.RandomState(seed))
    for t in range(N_RANDOM + N_BO):
        done = {r["params"]["trial"]: r for r in trials_so_far(T)}
        if t in done:
            continue
        hist = [done[i] for i in range(t)]
        if t < N_RANDOM:
            u = U0[t]
        else:
            X = np.array([r["params"]["u"] for r in hist]); y = np.array([r["summary"]["cv_loss"]
                for r in hist])
            C = np.random.RandomState(seed * 1000 + t).rand(CANDIDATES, len(space))
            u = C[d169.propose(X, y, C)[0]]
        p = dict(step="trial", trial=t, u=[float(x) for x in u], **d168.decode(u[None, :],
            space)[0])
        with T.run(p, seed, data, code) as r:
            if t == crash_at:
                raise RuntimeError(f"simulated crash in trial {t}")
            m, s = cv_score(D, p, folds, seed)
            r.summary.update(cv_loss=m, folds=s.tolist())
```

ACTUAL OUTPUT

```
learning-rate finder (Day 170): min / 10 = 2.18e-02; learning rates searched from 7.26e-05 to 6.53e-02
best 3-fold CV log loss after the 8 random trials 0.3439; after all 20 trials 0.3435
chosen: trial 14 (Bayesian), lr 2.97e-03, patience 20, batch 33
final fit on all 3000 tuning rows: stopped at best epoch 2; validation log loss 0.3358; test log loss 0.3331, test
    accuracy 0.8421
final weights fingerprint 770840edcfac; 22 tracked steps; 58 s in a new process (exit code 0)
first draft (rates from 7.26e-04): chose lr 4.02e-02, patience 12, batch 167; best CV 0.3413 against 0.3435; best epoch 1;
    validation 0.3426, test 0.3401
```

What it means

The study chose a Bayesian trial: trial 14, lr 2.97e-03, patience 20, batch 33. The optimiser improved the best CV score only a little over the random trials (0.3439 to 0.3435).

The final model: validation log loss 0.3358, test log loss 0.3331, test accuracy 0.8421. Early stopping kept epoch 2 of the final fit.

The first draft searched a narrower range, from 30 times below the suggestion (7.26e-04). It chose lr 4.02e-02, whose final fit peaked at epoch 1, and validation 0.3426, test 0.3401. Its CV score was better (0.3413): CV preferred the draft. I widened the range because a model that peaks after one epoch and Day 173's best rates near 1e-4 both said the range sat too high. **I had seen the draft's validation and test numbers when I made that change**, so this test score is not free of choices made after looking. The draft is kept as study D30 and its numbers are printed every run.

A likely reason the finder's range was too high, not tested here: it watches 300 steps of one run, while each trial may train for up to 100 epochs with early stopping, where smaller rates have time to pay off.

WHY CHOOSE THIS?

Composing tested parts means each piece was checked on its own first; the log ties them together.

WHY NOT THIS? (TRADEOFFS)

A part's assumptions travel with it: the finder's rule was set for short training, not for this budget.

2. The same study again, from scratch, in a new process

DEFINITION — start here, no prior knowledge assumed

Study B is the same command as A, in a new Python process. Every step's settings and every score are compared with A's to the last bit (== on the floats as written in the log).

REAL-WORLD SCENARIO

Baking the same recipe a second day and weighing both loaves to the milligram.

ACTUAL OUTPUT

```
21 of 21 steps identical to the bit (20 trials and the final fit); exit code 0, 57 s
trial 0: A 0.345463331144936, B 0.345463331144936
trial 12: A 0.3471156054791289, B 0.3471156054791289
final weights: A 770840edcfac, B 770840edcfac; run ids identical: True
```

What it means

21 of 21 steps identical, for example trial 0 at 0.345463331144936 in both. The final weights' fingerprint was 770840edcfac in both, and so were all run ids, since they hash the settings, seed, data and code.

What made this possible: every random number comes from a seeded `RandomState` (the finder, the random trials, the optimiser's candidates, the folds, the network's start and its batch order); nothing reads the clock or the global random state; and the thread count was fixed. Section 4 removes these one at a time.

3. A study that crashes in its Bayesian phase, then resumes

DEFINITION — start here, no prior knowledge assumed

Study C crashes in trial 13. The same command without the crash then resumes from the log: finished trials are skipped, and each Bayesian trial is proposed from the trials **before it**, read from the log.

REAL-WORLD SCENARIO

A night shift that picks up from the day shift's notes and builds exactly what the day shift would have built.

IMPLEMENTATION

```
        done = {r["params"]["trial"]: r for r in trials_so_far(T)}
        if t in done:
            continue
        hist = [done[i] for i in range(t)]
```

ACTUAL OUTPUT

```
first attempt: exit code 1, last error line: RuntimeError: simulated crash in trial 13
the log after the crash: 14 finished steps (the finder and trials 0 to 12), 1 failed
resumed: exit code 0, 23 s; 21 of 21 steps identical to study A, including the final weights 770840edcfac
```

What it means

After the resume, 21 of 21 steps matched the uninterrupted study, including the final weights (770840edcfac).

The first draft failed this check. It skipped a trial only when one with the same settings had finished, and gave the optimiser every finished trial. On resume the log already held trials 8 to 12, so trial 8 was proposed from a history that included its own future: different settings. That draft reproduced 13 of 21 steps and ended with different final weights (afed2a50971e), and it raised no error. Only comparing against an uninterrupted run found it.

WHY CHOOSE THIS?

A study that resumes from its own log can be stopped and restarted at no cost.

WHY NOT THIS? (TRADEOFFS)

Resume logic is easy to get subtly wrong; test it by crashing on purpose and comparing.

4. What breaks it: another seed, an unseeded fold split, one BLAS thread

DEFINITION — start here, no prior knowledge assumed

S1 changes the study seed from 0 to 1. U1 and U2 are the same command, run twice, with the fold split left unseeded. T1 is study A with one BLAS thread instead of four. For each: how many of A's steps were reproduced exactly, how many had A's exact settings, and the largest score difference among those.

REAL-WORLD SCENARIO

Changing one thing in the kitchen at a time to find which one changes the bread.

ACTUAL OUTPUT

```
S1 (study seed 1 instead of 0): 0 of 21 of A's steps reproduced; chose lr 9.84e-04, patience 3, batch 39; test log loss
    0.3364 (A 0.3331), weights 4e81f5f33d1c
    steps with A's exact settings: 1 of 22; largest score difference among them: none shared
U1 (folds with no seed, first run): 0 of 21 of A's steps reproduced; chose lr 1.60e-03, patience 7, batch 47; test log
    loss 0.3315 (A 0.3331), weights 336b47ebb101
    steps with A's exact settings: 9 of 22; largest score difference among them: 1.2e-02
U2 (folds with no seed, second run): 0 of 21 of A's steps reproduced; chose lr 1.08e-03, patience 16, batch 110; test log
    loss 0.3338 (A 0.3331), weights 673600440f24
    steps with A's exact settings: 9 of 22; largest score difference among them: 9.6e-03
T1 (seed 0, 1 BLAS thread instead of 4): 0 of 21 of A's steps reproduced; chose lr 2.97e-03, patience 20, batch 33; test
    log loss 0.3331 (A 0.3331), weights 770840edcfac
    steps with A's exact settings: 22 of 22; largest score difference among them: 4.6e-07
U1 against U2: 0 of 21 steps identical; the same command gave two different studies
```

What it means

The seed is part of the result. Seed 1 chose lr 9.84e-04, patience 3, batch 39, and a test log loss of 0.3364 against 0.3331. That gap is about eight times what the Bayesian trials gained over the random ones in section 1, so one study's choice says little about which settings are best.

A missing seed makes a command irreproducible. U1 and U2 tried the same 8 random configurations as A (their settings share A's seed), but with different folds their scores moved by up to 1.2e-02 and 9.6e-03, as large as most differences between A's trials. U1 against U2: 0 of 21 identical, choosing lr 1.60e-03 and 1.08e-03. Rerun the script and U1 and U2 change again; that is the point.

One BLAS thread changed every score, but only in the last digits. T1 had A's settings at 22 of 22 steps, the largest difference 4.6e-07, the same choice and the same final weights (770840edcfac). Bit-identical results hold for a fixed thread count, not across them. A likely reason the weights stayed identical while scores did not, not tested here: training multiplies 33-row batches, which the library may not split across

threads, while scoring multiplies thousands of rows at once, which it does, adding the pieces in a different order.

WHY CHOOSE THIS?

Seeding every random source and fixing threads made the study bit-identical on this machine.

WHY NOT THIS? (TRADEOFFS)

Bit-identical is fragile across machines and libraries; for comparisons, measure the seed spread instead.

5. Pinning: the environment recorded and checked, and a line-ending trap

DEFINITION — start here, no prior knowledge assumed

The first run writes `day174_environment.json` (Python, numpy and matplotlib versions, machine, system, BLAS threads, hash seed) and `day174_requirements.txt`; later runs compare themselves with the pin. Then the script hashes its own bytes three ways.

REAL-WORLD SCENARIO

A recipe card that names the brand of flour and the oven's model, not just 'flour' and 'oven'.

ACTUAL OUTPUT

```
  wrote day174_environment.json
  pinned: blas_threads 4, machine AMD64, matplotlib 3.11.0, numpy 2.5.0, python 3.14.2, pythonhashseed 0, system Windows
  this machine against the pin: no differences
  day174_requirements.txt: numpy==2.5.0, matplotlib==3.11.0 (python 3.14.2)
  every start record carries its environment too, e.g. study A's first: {'numpy': '2.5.0', 'python': '3.14.2'}
  code hash of this script on disk e301653f65b9 (the hash every record carries); with LF line endings e301653f65b9, with
      CRLF line endings 048605463d57: the same code, two hashes
```

What it means

Every start record also carries Python and numpy versions, so even a log copied elsewhere says where it came from. The pin matched (it was written by this run: no differences).

The line-ending trap. The script on disk hashes to e301653f65b9, the hash every record carries. The identical code with Windows line endings (CRLF) hashes to 048605463d57. This repository is checked out on Windows with Git's core.autocrlf setting on, which converts line endings, so a learner rerunning this study there could see a different code hash for the same code. The fix is to hash the text with line endings normalised, or to tell Git to keep .py files as LF (a .gitattributes rule).

WHY CHOOSE THIS?

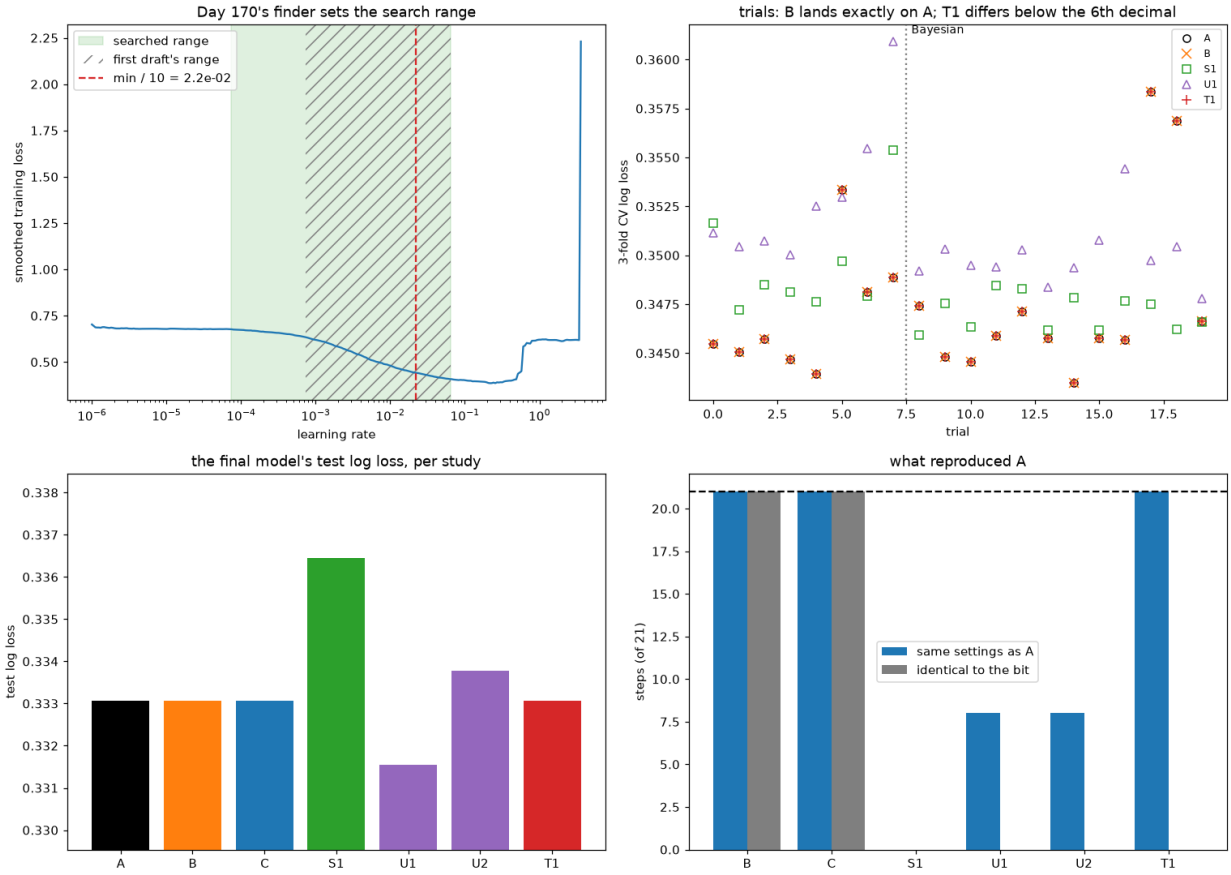
Pin files are cheap and turn 'it worked on my machine' into a checkable difference.

WHY NOT THIS? (TRADEOFFS)

Pinning versions does not pin hardware or BLAS builds; containers go further, at more cost.

Week 25 in one table

Day	Part	What it did in this build
168	random_unit, decode, best_so_far	8 random trials on log, whole-number and log-whole-number scales
169	propose (GP + expected improvement)	12 trials proposed from the trials before them
170	lr_find, suggest	set the learning-rate range (min / 10 = 2.18e-02); its range was too high at first
171	EarlyStopper, train_es	chose the epoch of every fold and of the final fit
172	shuffled_folds, cross_val	3-fold scores; shuffled is right here, Adult rows are separate people
173	Tracker, fingerprint, run_id	logged every step; the optimiser and the resume read from it



Top left: the finder's curve, the searched range and the first draft's narrower one. Top right: every trial's CV score in A and its variants. Bottom left: the final test log loss of each study. Bottom right: steps with A's exact settings, and steps identical to the bit.

Interview preparation — technical questions

Q: How do you make a training run bit-identical?

A: Seed every random source explicitly, avoid global and time-based randomness, fix thread counts and library versions; here 21 of 21 steps matched.

Q: How do you test that a resumable study really resumes correctly?

A: Crash it on purpose and compare with an uninterrupted run; a first draft here passed silently and reproduced only 13 of 21 steps.

Q: Why might the same seed give different results on another machine?

A: Different BLAS builds or thread counts add numbers in different orders; one thread here changed scores by up to 4.6e-07.

Q: What goes in a pinned environment?

A: Exact package versions, Python version, and settings that change results such as thread counts and hash seeds.

Q: How big is seed noise compared with tuning gains?

A: Here seed 1 moved the test loss by 0.0033, about eight times what Bayesian trials gained over random ones.

Interview preparation — theoretical questions

Q: Bit-identical or statistically reproducible: which do you need?

A: Bit-identical for debugging and audit; statistical, with measured seed spread, for claims about which model is better.

Q: Why can a code hash change when the code does not?

A: It hashes bytes; line endings or encoding change bytes without changing meaning. Normalise before hashing.

Q: Is reporting a test score after changing the search valid?

A: Only with the change disclosed; once the test was seen, the score is no longer an untouched estimate.

Q: What does a seeded but unrecorded study lose?

A: Everything: without the seed in the log, nobody can rerun it, including you.

Key takeaway and what's next

> **KEY TAKEAWAY**
>
> Reproducibility is a property to test, not to assume. Run the same command twice in fresh processes and compare every number; crash it and resume; change one source of randomness at a time. Here that found a silent resume bug, showed a missing seed making one command two studies, and showed that bit-identical results hold only for a fixed thread count.

- Same command, new process: 21 of 21 steps identical; final weights 770840edcfac both times.
- Crash and resume: identical after fixing a first draft that reproduced 13 of 21.
- Seed 1: test 0.3364 against 0.3331; unseeded folds: two different studies from one command.
- One BLAS thread: same choices and weights, scores changed by up to 4.6e-07.

Suggested watch order, if starting from zero

- 1. Reproducible machine learning and science with python (Prabhant Singh, PyData) — reproducible machine learning and science with Python.
- 2. Using Reproducible Experiments To Create Better Machine Learning Models (PyData Global 2021) — running reproducible experiments to build better models.

Day 175 preview

- **Week 26, MLOps I: serving models.** Day 175 compares real ways of saving and loading a trained model, starting from a model like this one, whose weights must load back identical to the bit.